Chapter 5
Arrays and Text Files

This chapter introduces arrays, reading and writing text files, as well as parsing data from text files. While the chapter uses examples to demonstrate these key techniques, a number of related programming skills will be discussed, which include resizing arrays, array properties and methods, string splitting, error handling, and the open file dialog.

1. Array

So far you have been using one variable to store one value at a time. If a class has five students, you may have to declare five variables to store all students' names. It is a tedious work to declare many similar variables and assign values to them one by one. What about if you have to handle 100 student names? What if you have to read student names from a file in which the number of students is unknown beforehand? Anyway, declaring a series of similar variables to store a long list of similar values does not make sense. Besides that, with tons of variables in a procedure, the code is cumbersome and inefficient. A solution to this kind of problem is to use arrays.

An array is a variable that can hold a series of values of a same type. Declaring an array variable is similar to declaring a single value variable with the following syntax:

```
Dim arryayName([upperBound]) As DataType
```

For example, to create an array for holding five student names, you can use code

```
Dim Students(4) As String
```

This line of code declares a String type array variable named *Students*. The number (4) in the parentheses is the upper bound of the array. The actual size of the array is the upper bound + 1 because indexes in VB.NET always start from 0. After declaring the array variable *Students*, you can store student names in it, i.e. to assign values to elements of the array variable. For example,

```
Students(0) = "Andria"
Students(1) = "Bryan"
Students(2) = "Carrie"
Students(3) = "Daniel"
Students(4) = "Eva"
```

In the computer, the array occupies a continuous block of memory space (Figure 1)

Students(0)	Students(1)	Students(2)	Students(3)	Students(4)
Andria	Bryan	Carrie	Daniel	Eva

Figure 1 Values stored in an array

Like single value variables, arrays can also be declared and initialized (assigned values) in one line. For example, the following line does the same job as the six lines above.

```
Dim Students() As String = {"Andria", "Bryan", "Carrie", "Daniel", "Eva"}
```

But when you declare an array variable and initialize it in one line, you must not specify an upper bound in the parentheses. The size of the array is determined by the number of values in the curly brackets. Please also note that initial values must be separated by commas, and String values must be quoted by double quotation marks.

An array is an object so it has properties such as Length and methods such as Count(). Both the Length property and the Count() method tell the number of elements contained in the array. To distinguish a method from a property, a pair of parentheses is usually placed behind the name of a method by convention. If an array is a numeric data type, it also has a number of methods that perform some basic statistics on elements of the array. These methods include Sum(), Max(), Min(), Average(), etc. To find properties and methods of an array, you can type a dot (.) after an array variable. IntelliSense immediately lists commonly used properties and methods of the array next to it (Figure 2). You can click the "All" tab to see all properties and methods. Properties are labelled by the icon and methods are labelled by in IntelliSense.

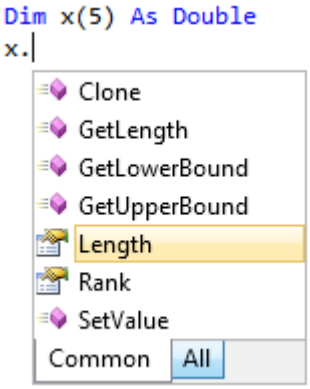


Figure 2 Properties and methods of arrays displayed in IntelliSense

2. Reading Text Files

VB provides a variety of functions to read text files. Function IO.File.ReadAllLines(<file path>) is an easy one to use. Given the full path of a text file, this function reads all content of the file and returns a String type array in which each element holds a line in the file (Figure 3). In a text file each line must be ended by a line break (carriage return) which is an invisible special character. Therefore, the number of lines in the text file determines the numbers elements in the resulting array.

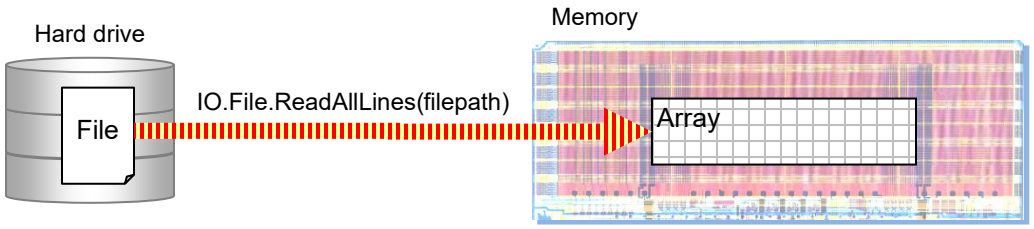


Figure 3 Reading the contents of a file on the hard drive into an array in the memory

The function name consists of three components separated by dot (.) symbols. In fact, the first two words "IO" and "File" are namespaces. To put it simple, a *namespace* is a tree-like hierarchical structure (like Windows folders) used by a programming language to organize its functions and objects. Since function ReadAllLines() exists in namespace File under namespace IO, the full path to this function is referenced by IO.File.ReadAllLine(). In fact, you have seen functions referenced with namespaces such as Math.Round(), Math.Sqrt(), and Math.Pow(). Since these functions exist in namespace "Math", they have to be referenced with the "Math" namespace. Namespace will be discussed in detail in Chapter 6.

Function ReadAllLines() has a mandatory String type parameter *filepath* which is used to receive the full path of the text file to open. Therefore, when you use this function to open a text file, you must provide the full path of the file to the function.

Example 1: Create a program to read student names from text file "Students.txt" (in folder "Data" on the CD-ROM). Once started, the main form's Load event calls a data loading sub procedure LoadData() to open the text file and reads all student names into a class-level array variable. Then the data loading procedure uses a repetition to add each name into a list box. The program should also provides a text box for the user to enter an index number and a button that displays a name of a given index in a message box when clicked. Use the following settings to design the project GUI.

Control	Name	Purpose
Form	frmMain	The main form
TextBox	txtIndex	For receiving user input name index
Button	btnShow	To show student name of given index
ListBox	lstStudents	To display student names
Label	lblCount	To display the total number of students

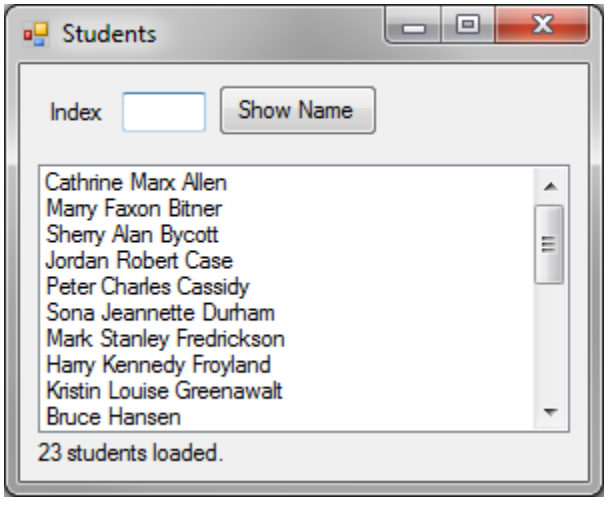


Figure 4 Project GUI and running result of Example 1

```
Public Class frmMain
    ' declare a class-level array variable to hold student names
    Private Students() As String

    Private Sub Form1_Load(...) Handles MyBase.Load
        LoadData()    ' calls LoadData() once the program starts
    End Sub

    Private Sub LoadData()
        ' call the ReadAllLines() function to read the text file and
        ' assign its output to array variable Students
        Students = IO.File.ReadAllLines("D:\Data\Students.txt")

        ' use a loop to add elements in the array into the list box
        For i As Integer = 0 To Students.Length - 1
            lstStudents.Items.Add(Students(i))
        Next

        ' display student count in a label
        lblCount.Text = CStr(Students.Length) & " students loaded."
    End Sub

    Private Sub btnShow_Click(...) Handles btnShow.Click
        ' get index entered by the user
        Dim index As Integer = CInt(txtIndex.Text)

        ' display a name of the specified index
        MessageBox.Show(Students(index), "Student",
                        MessageBoxButtons.OK, MessageBoxIcon.Information)
    End Sub
End Class
```

In this program, the first line inside the class block declares a class-level String-type array variable *Students*. This variable will get values in the LoadData() procedure which is called by the main form's Load event procedure. In the LoadData() sub procedure, the first line of code

```
    Students = IO.File.ReadAllLines("D:\Data\Students.txt")
```

calls the built-in function ReadAllLines() to read the content of the text file specified in the parentheses. Please make accordingly change to the file path to reflect the actual file location on your computer. The output of the function, which is an array containing all student names read from the file, is then assigned to the array variable *Students*. After reading the text file, a For … Next repetition is used to add each element of the array into the list box. Because the LoadData() sub procedure is called in the form Load event, all these actions occur before the main window opens. In the repetition block, `Students.Length - 1` is the upper bound of the *Students* array because array indexes start from 0. The last line in the LoadData() sub procedure displays the total number of students as a label.

3. Handling errors

The program of Example 1 should start up fine if a text file with the specified path exists. However, Murphy's Law (which states "if anything can go wrong, it will") seems to favor programming over anything else. For example, if you click the "Show Name" button when the text box is empty, or you happen to enter a non-numeric character, or you enter a number that exceeds the index range of the *Students* array (0 to upper-bound), the program will crash.

To avoid crashing, VB.NET provides a Try block to capture exceptions (run-time errors) in programs. The Try block syntax is:

```
Try
    ' put some code in this block to try
Catch ex As Exception
    ' if anything goes wrong in the try block, an error is raised
    ' and the program flows into this block
    'for the programmer to handle the error
End Try
```

To avoid crashing caused by potential invalid user input, you can use a Try block in the Click event procedure of the button (btnShow) as the following:

```
Try
    Dim index As Integer = CInt(txtIndex.Text)
    MessageBox.Show(Students(index), "Student",
                    MessageBoxButtons.OK, MessageBoxIcon.Information)
Catch ex As Exception  ' ex contains error info if an error is caught
    MessageBox.Show(ex.Message)
    Return
End Try
```

This Try block can capture any error that may occur to the two lines of code in the Try block. For example, if the text box is empty or the user enters an invalid number, the CInt() function will go wrong since it cannot convert a blank or invalid string into an integer number. If an error is raised in the first line, the program flow will be diverted to the Catch block to handle the error. If the first line is executed successfully, a resulting integer number is assigned to variable *index*. However, if the index value is outside the bounds of array *Students*, the second line will go wrong because "Students(index)" cannot be evaluated, and the program will also be directed to the Catch block.

In the Catch block, parameter *ex* (that follows the Catch keyword) receives the error information. Inside the Catch block, the error is handled by sending a message box to display the error information, and then using the Return statement to force the program to exit the procedure, i.e. to flow to the End Sub statement.

In Example 1, other chances for the program to crash badly include to pass a non-existing file to the *ReadAllLines*() function or the file is not a text file. If either of these errors occurs, when you click the run (debug) button, the program will crash before you even see the main form because the LoadData() procedure which contains the ReadAllLines() function is called by the form Load event. To avoid the problem, you can also put the code susceptible to errors into a Try block.

```
Try
    Students = IO.File.ReadAllLines("D:\Data\Students.txt ")
Catch ex As Exception
    MessageBox.Show(ex.Message)
    Return
End Try
```

Although the Try structure can capture run-time errors, it does not capture logical errors since they are defects in methodology or algorithms. Even with run-time errors, the information contained in parameter *ex* is often general or vague. Good programmers should be able to anticipate possible errors and use custom code to preclude them. Instead of using the Try block, for example, you can use your own code to validate user input in the Click event procedure of the show name button (btnShow). Please pay attention to the comments when you read the code below.

```
Private Sub btnShow_Click(...) Handles btnShow.Click
    ' to make the program robust, you need to validate the user input.
    ' first, check if the text box is empty. If so, exit the procedure
    If txtIndex.Text = "" Then Return

    ' second, check if the user input is a number
    If Not IsNumeric(txtIndex.Text) Then
        MessageBox.Show("Index is not a number.", "Error",
                        MessageBoxButtons.OK, MessageBoxIcon.Error)
        Return
    End If

    ' third, convert user input into integer and store it in a variable
    Dim index As Integer = CInt(txtIndex.Text)

    ' finally, check if the index value is within bounds of the array
    If index < 0 Or index > Students.Count - 1 Then
        MessageBox.Show("Index exceeds array bounds.", "Error",
                        MessageBoxButtons.OK, MessageBoxIcon.Error)
        Return
    End If

    ' at this point, everything should be good. You can display the name
    MessageBox.Show(Students(index), "Student",
                    MessageBoxButtons.OK, MessageBoxIcon.Information)
End Sub
```

After modifying the code as above, your program should be able to handle any situation and never crash. In the above event procedure, IsNumeric() is a VB built-in function that checks if a value is a number. This function returns a Boolean value.

4. Open File Dialog

Although after handling potential errors the program of Example 1 is crash proof, the program is rigid because it can only open one file whose path is hard coded in it. To

increase flexibility, this section will add a capability that allows the user to open a file dialog box and browse a file to open.

To implement this capability, please add a text box (name it "txtFile") and a button (name it "btnBrowse") on the main form. Please also put a label ahead of the text box to indicate the purpose of the text box (Figure 5). We hope that when the user clicks the browse button, a file browsing dialog opens. Once the user selects a file in the dialog, the file name appears in the text box, the contents of the file is loaded into the array variable and also displayed in the list box.

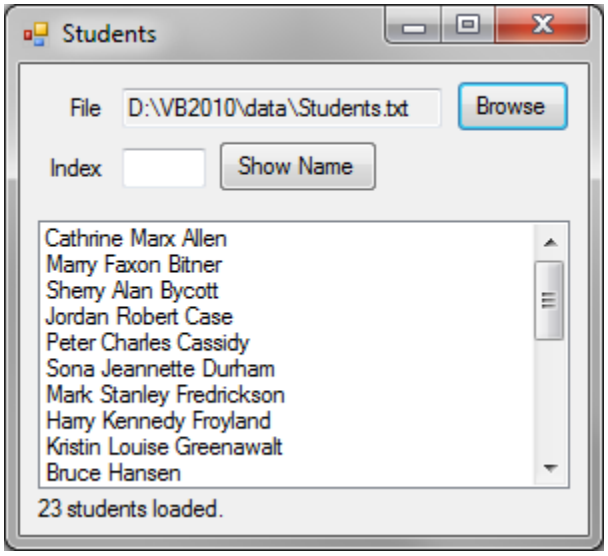


Figure 5 Improved user interface of the Student program

Before making more changes, please first delete or comment out the line in the form's Load event procedure that calls the LoadData procedure. You will call the LoadData procedure after a file is selected by the user.

In VB, a file browser that lets the user to select a file is an object called "open file dialog". This object does not exist by default, and you have to create it. In object-oriented programming, an object can be created from a class which is a template for a certain type of objects. VB provides a built-in class named OpenFileDialog which can be used to create open file dialog objects.

In the browse button's Click event procedure, please use the following code to create an open file dialog object named "dlgFile" from the OpenFileDialog class.

```
Dim dlgFile As New OpenFileDialog()
```

Creating an object from a class is similar to declaring a variable from a simple data type except that it uses a required keyword New. The pair of parentheses after the class name is optional if the class constructor does not require arguments. More about creating objects from classes is introduced in Chapter 6. After creating a dialog object,

you usually need to set a few properties to make it user friendly and look nice. For example, you often need to set a title to let the user know the purpose of this dialog. You can also set a file filter so that only the specified file types will show up in the dialog box.

```
dlgFile.Title = "Please select a student file"
dlgFile.Filter = "Text File *.txt|*.txt"
```

The first line sets a title “Please select a student file” to the dialog, and the second line sets a file filter that only allows text files to show in the dialog box. A file filter is string containing the following components.

```
"File type description *.extention | *.extention"
```

A filter string can define multiple file types which are separated by vertical bars. For example, if you want to display text file and Comma Separated Values (CSV) files, you can use the following filter:

```
dlgFile.Filter = "Text File *.txt|*.txt|CSV File *.csv|*.csv"
```

To show the dialog box, you can call the ShowDialog() method of the dialog object, i.e.

```
dlgFile.ShowDialog()
```

Once the user selects a file and clicks the Open button, the full path of the file can be obtained from the FileName property of the dialog object. For example, you can use the following code to get the user selected file and display its full file path to the text box.

```
txtFile.Text = dlgFile.FileName
```

Once the open file dialog object (dlgFile) is no long needed, you can call the Dispose method of object to destroy it and recycle memory.

```
dlgFile.Dispose()
```

It is a good practice to always destroy unneeded objects created in a procedure using your own code although VB has a housekeeping capability.

Finally, the file browsing button’s Click event procedure should look like the following:

```
Private Sub btnBrowse_Click(...) Handles btnBrowse.Click
    ' create a new OpenFileDialog object
    Dim dlgFile As New OpenFileDialog()

    ' configure the open file dialog object
    dlgFile.Title = "Select a student file"
    dlgFile.Filter = "Text File *.txt|*.txt|CSV File *.csf|*.csv"

    ' open the open file dialog for the user to select a file
    dlgFile.ShowDialog()

    ' display the file name in the text box
```

```
        txtFile.Text = dlgFile.FileName

        ' dispose the open file dialog to release memory
        dlgFile.Dispose()
    End Sub
```

Now, you can run the program to test the file browsing button. Once you select a file and click the Open button in the open file dialog box, the full path of the file should appear in the file name text box, but nothing else occurs.

Stop running the program and add the following code to the end of the file browsing button's Click event procedure, just above the closing statement.

```
    Private Sub btnBrowse_Click(...) Handles btnBrowse.Click
        ' other code ...
        LoadData()    ' calls the LoadData() procedure
    End Sub
```

Then in the LoadData procedure, replace the hard coded file path by txtFile.Text, i.e. to pass the text in file text box (txtFile) to function ReadAllLines() as an argument, so that the function will read the file specified in the text box.

```
    Try
        Students = IO.File.ReadAllLines(txtFile.Text)
    Catch ex As Exception
        MessageBox.Show(ex.Message)
        Return
    End Try
```

After making all the above changes, start the program again to test the file browsing capability. Once you select a student file and click the Open button, the file will be read and student names will be displayed in the list box.

5. ReDim and Preserve

Keyword ReDim is used to resize an array. For example,

```
Dim num(1) As Integer  ' declares array variable num to hold two values
num(0) = 10
num(1) = 15
ReDim num(4)           ' resizes the num array for holding five values
```

When you use keyword ReDim to resize an array, 1) existing values in the array will be lost. With the above example, if you check the values of num(0) and num(1) after resizing the array, you will find both are 0; 2) You cannot specify a data type again when you use ReDim to resize an array because the keyword cannot change the data type of an array. Therefore, "`ReDim num(4) As Integer`" is a wrong statement.

Keyword Preserve is used to preserve existing values of an array when you resize it. For example,

```
ReDim Preserve num(4)
```

This increases the size of array variable *num* to 5 while preserving any existing values. Keywords ReDim and Preserve can be used for either expanding or shrinking arrays.

Example 2: Create a new project named "EnterNumbers" to allow the user to enter a series of integer numbers in a text box (txtNumber). Each time a number is entered and the Enter key is pressed, the number is added into a list box (lstNumbers) and also stored in an array variable, then the text box is cleared for new entries. When you click on a "Sort" button (btnSort), the values in the array are sorted in ascending order and the list box is updated (Figure 6).

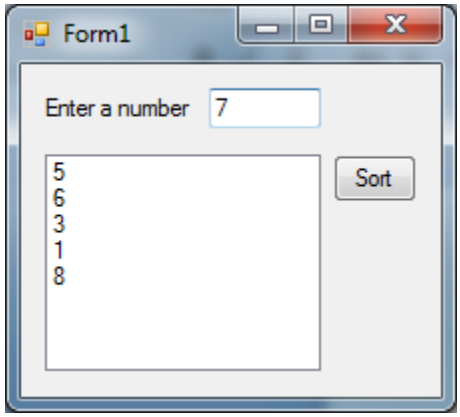


Figure 6 User interface and running result of Example 2

You may have noticed that an event procedure of a control normally has two arguments, *sender* and *e*. So far we have never used these parameters although we use event procedures in every VB program. Parameter *sender* holds the control object that fires the event, and parameter *e* carries information of the interaction. For example, if the event is MouseDown of a Windows form, parameter *sender* refers to the form object and parameter *e* contains the mouse location and mouse button information (which button is pushed). In this example, we will need to use the KeyDown event of the text box control. In this event procedure, parameter *e* will contain the key code information so that the program knows which key is pushed when the event is fired.

This example also lets you to practice several other programming skills including the use of keywords, ReDim, Preserve, ByRef, swap numbers, and repetitions. Please carefully read through the following code and comments before implementing your project.

```
Private num() As Integer      ' declares a blank class-level array for numbers
Private index As Integer = 0  ' declares a class-level integer for index

Private Sub txtNumber_KeyDown(...) Handles txtNumber.KeyDown
    If e.KeyCode = Keys.Enter Then              ' if Enter key is pressed
        If Not IsNumeric(txtNumber.Text) Then   ' validate user input
            MessageBox.Show("Invalid number.", "Error",
                            MessageBoxButtons.OK, MessageBoxIcon.Error)
            Return    ' if the user input is not a number, exit the procedure
        End If
```

```
            ReDim Preserve num(index)           ' resizes the num array
            num(index) = CInt(txtNumber.Text)   ' stores the number in the array
            lstNumbers.Items.Add(txtNumber.Text) ' adds the number to the list box
            txtNumbers.Clear()                   ' clears the text box
            index += 1                          ' finally, increments the index
        End If
    End Sub

    Private Sub btnSort_Click(...) Handles btnSort.Click
        SortArray(num)      ' calls the SortArray sub procedure
        DisplayNumbers()    ' calls the DisplayNumbers sub procedure
    End Sub

    Private Sub SortArray(n() As Integer)
        For i As Integer = 0 To n.Length - 2
            For j As Integer = i + 1 To n.Length - 1
                If n(i) > n(j) Then Swap(n(i), n(j))
            Next
        Next
    End Sub

    Private Sub Swap(ByRef a As Integer, ByRef b As Integer)
        Dim tmp As Integer = a
        a = b
        b = tmp
    End Sub

    Private Sub DisplayNumbers()
        lstNumbers.Items.Clear()
        For i As Integer = 0 To num.Length - 1
            lstNumbers.Items.Add(num(i))
        Next
    End Sub
```

The class-level array variable *num* is declared without initially specifying an upper bound. Therefore it cannot hold any value until a non-negative integer is specified as an upper bound by using keywords ReDim Preserve. In the text box's KeyDown event procedure, if the "Enter" key is pushed (`e.KeyCode = Keys.Enter`), a number of operations are performed. These operations include validating the user input, expanding the class-level array variable *num*, storing the new value in the array, adding the number into the list box, and incrementing the index for use in next round of data entry.

The Click event of the sort button simply calls two sub procedures, SortArray() and DisplayNumbers(), to carry out tasks. While the DisplayNumbers() function is straightforward, the SortArray() function is explained in detail below.

First, the SortArray() sub procedure declares one array parameter n() of Integer type. The pair of parentheses behind parameter name indicates that the parameter is an array. This parameter is used to receive an input array of integer values to sort.

Second, please recall that by default, argument passing is by value if no ByVal or ByRef specified to the parameter. But this rule only applies to numeric (e.g. Integer, Double, etc.), String, and Boolean type parameters. It is not true for arrays and object-type parameters. When a parameter of a procedure is an array type, no matter whether it is declared with ByVal or ByRef, it is always treated as ByRef. This means that if a

procedure changes element values in the array parameter, the changes are actually made in the original array in its client procedure. This is what we exactly want in this program.

Third, the sort button's Click event calls the SortArray() sub by passing array *num* to it. Then address of *num* is received by parameter *n* of procedure SortArray(). When elements of parameter *n* are sorted in SortArray(), elements in array *num* in the button Click event procedure are also sorted.

Finally, sorting number in an array involves a series of swapping of elements in the array. To sort in ascending order, the procedure compares an element with each element behind it in the array. If the element is greater than the one behind it, they are swapped. When all pairs of elements are compared and swapped when necessary, the array is sorted. Please analyze the two nesting *For* … *Next* repetitions in sub procedure SortArray() to figure out how the sorting number algorithm works. You are suggested to use debugging skills to step through the double loop and watch when values in the array are exchanged by the Swap() procedure. Please recall that swapping individual values is discussed in Chapter 4.

6. The Split method of string objects

Strings in VB are objects. String objects have methods such as ToUpper(), ToLower(), Trim(), IndexOf(), LastIndexOf(), SubString(), etc. This section introduces another useful method of strings – Split(). This method splits a string by a specified delimiter (separator). The result of this method is a String type array containing values split from the original string. The delimiter can be a character or a string. For example,

```
Dim str As String = "FID,ROUTE,UTC,LAT,LON,ALT,SATS,SATV"
Dim data() As String = str.Split(","c)
```

In the code above, the delimiter `","c` is a comma defined as a character. The result of the Split() method is assigned to a new string array variable *data* declared in the same line. If you check the contents of the *data* array, you should find string values "FID", "ROUTE", "UTC", "LAT", "LON", "ALT", "SATS", and "SATV" are sequentially stored in elements *data*(0) through *data*(7).

This method is very useful, especially when it is used together with a text file reading function. The combination is popularly used to parse data from comma-separated value (CSV) files and other text files. For example, given a comma delimited text file ("busroutes.txt" folder "Data" on CD-ROM) containing bus route data recorded by a GPS, we can write a program to extract the coordinates of any specific route, and compute the length of any bus route.

Example 3: Five potential bus routes are recorded by GPS for transportation analysis. A Comma Separated Value (CSV) file ("busroutes.csv" in folder "Data" on CD-ROM) is exported from the GPS with the following structure.

```
FID,ROUTE,UTC,LAT,LON,ALT,SATS,SATV
1,1,"20/08/2004 02:03:52.000",35.1759733337826,-111.656433333291,2081.6,5,9
2,1,"20/08/2004 02:03:53.000",35.1759700001611,-111.656419999864,2081.5,5,9
3,1,"20/08/2004 02:03:54.000",35.1759616671668,-111.656396666633,2081.2,5,9
...
```

Write a program to display the geographic coordinates of any user-specified bus route for analysis.

A CSV file is a text file so it can be read by the ReadAllLines() function. The first line of the data file is a header that contains eight field names separated by commas. Field "FID" is a unique ID for each GPS point, field "ROUTE" contains bus route numbers, field "UTC" records the UTC time when each point was recorded, fields "LAT", "LON", and "ALT" record the latitude, longitude, and altitude values, and fields "SATS" and "SATV" record number of satellites tracked and number of satellites in view.

The flowchart below (Figure 7) describes a procedure that displays the latitude / longitude coordinates of a user-specified bus route from the file.

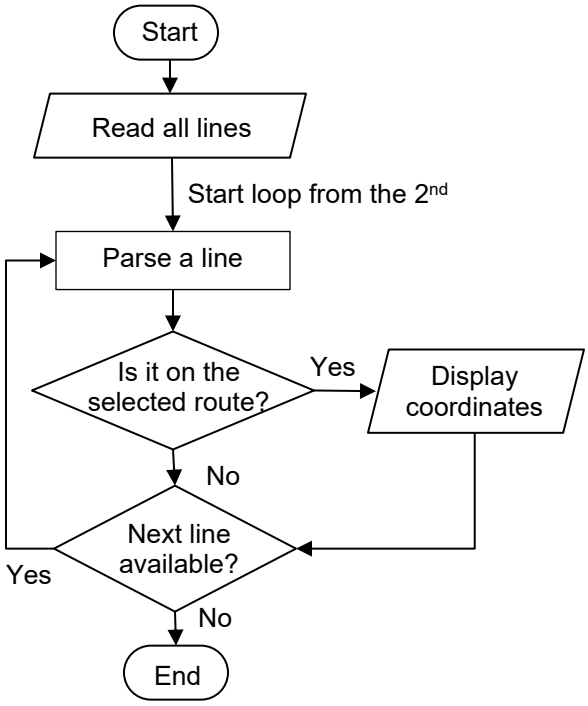
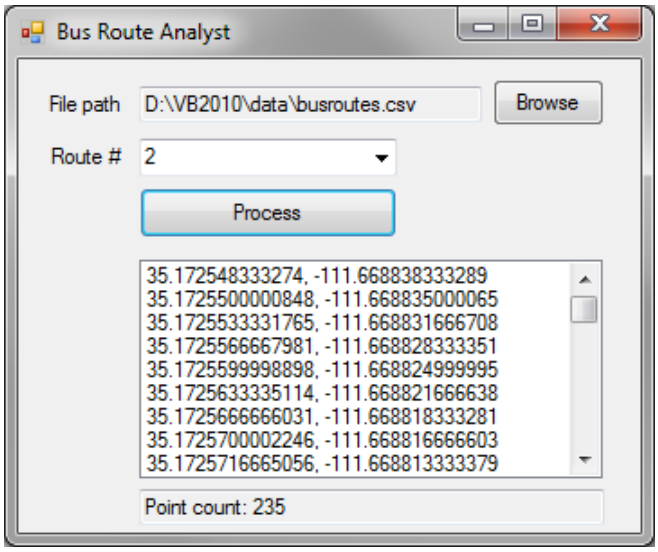


Figure 7 Flowchart for displaying coordinates of a selected bus route

Create a new VB project named "RouteCoordinates" and design the project GUI by referring to Figure 8. To implement the functionality, please perform the following tasks.

1) Declare a class-level array variable (allLines) that will be used to hold all lines of the data file.

```
Private allLines() As String
```



Control	Property Settings
Form	Name: frmMain; Text: Bus Route Analyst
TextBox	Name: txtFile ReadOnly: True
ComboBox	Name: cboRoute
Button	Name: btnProcess Text: Process
ListBox	Name: lstOutput
TextBox	Name: txtInfo ReadOnly: True

Figure 8 Application GUI of the bus route analyst project

2) In the form's Load event procedure, use a loop to add five bus route numbers 1 through 5 into the combo box.

```
Private Sub frmMain_Load(...) Handles MyBase.Load
    ' add numbers 1 through 5 into the combo box
    For i As Integer = 1 To 5
        cboRoute.Items.Add(i)
    Next
    cboRoute.Text = "Select a bus route"
End Sub
```

3) Implement the file browsing button (introduced in Section 4). In the end of the browse button's Click event procedure, call the ReadAllLines() function to read the data file.

```
Private Sub btnBrowse_Click(...) Handles btnBrowse.Click
    Dim dlgFile As New OpenFileDialog
    dlgFile.Title = "Select A Bus Route File"
    dlgFile.Filter = "CSV File *.csv|*.csv"
    dlgFile.ShowDialog()
    txtFile.Text = dlgFile.FileName
    dlgFile.Dispose()

    ' if a file is selected, read contents of the file into array allLines
    If txtFile.Text <> "" Then
        allLines = IO.File.ReadAllLines(txtFile.Text)
    End If
End Sub
```

4) Implement the process button's Click event.

```
Private Sub btnProcess_Click(...) Handles btnProcess.Click
    ' check if the file is loaded or not. if not, exit the sub
    If allLines Is Nothing Then Return

    Dim data() As String       ' an array variable for holding data of a line
```

```
    Dim count As Integer = 0   ' an integer variable used to count points

    lstOutput.Items.Clear()    ' clears the list box
    For i As Integer = 1 To allLines.Length - 1
        data = allLines(i).Split(","c)      ' parses a line
        If data(1) = cboRoute.Text Then     ' data(1) holds a route number
            ' display coordinate of a GPS point
            lstOutput.Items.Add(data(3) & ", " & data(4))
            count += 1      ' increments the counter
        End If
    Next
    ' display point count of the selected bus route
    txtInfo.Text = "Point count: " & CStr(count)
End Sub
```

7. StreamReader and StreamWriter

The ReadAllLines() function reads an entire text file and returns an array containing all lines of the file. So it can be memory demanding if the text file is large since it dumps the entire file into memory all at once. If a program only needs to process a subset of the data (e.g. selected rows or columns) in a large text file, the ReadAllLines() function may not be the best choice.

7.1 StreamReader

StreamReader is a VB.NET built-in class. It can be used to create stream reader objects that read text files line-by-line. In case we only need to process a subset of the data contained in a large text file, the StreamReader class can be more efficient than function ReadAllLines(). The StreamReader class exists in the IO namespace so it can be referenced by IO.StreamReader. The following code creates a new object from the StreamReader class.

```
Dim sr As New IO.StreamReader(<file path>)
```

Variable *sr* can be regarded as a stream reader object created from the StreamReader class. Once a stream reader object is created this way, the text file, whose name is passed to the class as an argument, is open for reading and the ReadLine() method of the object can be called to read a line at a time. Method ReadLine() returns a string and moves a pointer in the stream reader object *sr* to the next line. We can use a string variable to receive the output of the ReadLine() method. For example,

```
Dim str As String = sr.ReadLine()  ' reads a line and assigns it to str
```

To read through a text file, we can use a While repetition block to repeatedly call the ReadLine() method. The StreamReader class has a read-only property named EndOfStream that tells whether the pointer reaches the end of the file. Therefore, we can use the EndOfStream property to terminate the repetition when the pointer reaches the end of the file. For example,

```
While Not sr.EndOfStream
    Dim str As String = sr.ReadLine()
```

```
        ' process string str ...
    End While
```

Once a stream reader object (*sr*) is created, a specified text file is open and a lock is applied to that file to prevent it from being changed by other programs. Once your program finishes processing data in the file, you must use code close the file to remove the lock. The Close() method of a stream reader object does this job. For example,

```
sr.Close()
```

Example 4: To demonstrate the usage of the StreamReader class, we are going to use a different way to display coordinates of a selected bus route for the problem in Exercise 3. In this example, we will also add a new capability to calculate the length of the select bus route.

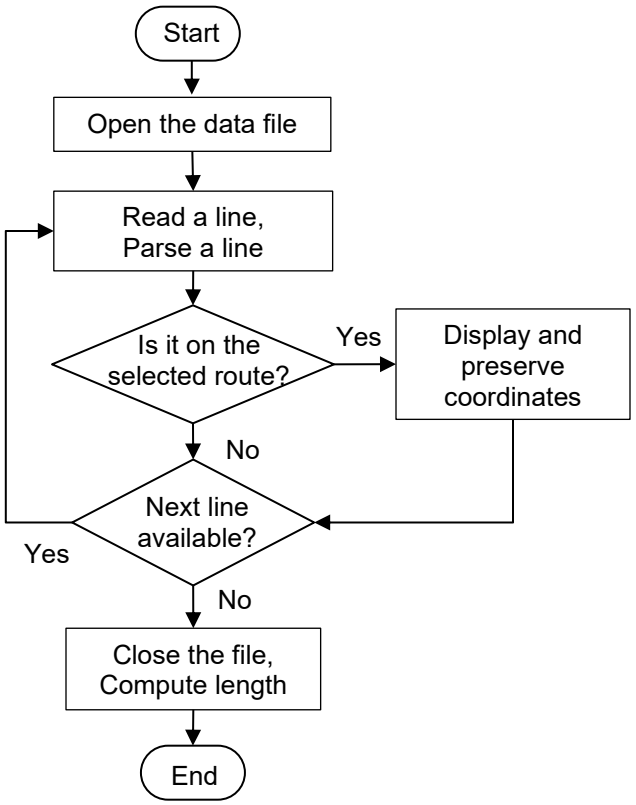


Figure 9 Flowchart for extracting coordinates and computing bus route length

Figure 9 describes an overall procedure for carrying out the tasks. This procedure will use a stream reader to open the file. Then it uses a repetition block to read one line from the file and parses data from the line at each iteration. If the line read belongs to the user selected route, the program displays its coordinates and preserves the coordinates in two array variables for later use. After iterating through all lines in the file, the program closes the data file and computes the length of the selected bus route.

To implement this procedure, please create a new VB project and name it "BusRouteAnalyst". Then design a same project GUI as project "RouteCoordinates" in Example 3 and perform the following tasks.

1) In the main form's Load event procedure, add five bus route numbers 1 through 5 into the combo box.
2) Implement the file browsing button's click event procedure. This event procedure only lets the user to browse a file and display it in the input file text box.
3) Use the following code to implement the process button's Click event.

```
Private Sub btnProcess_Click(...) Handles btnProcess.Click
    ' if no data file is selected, exit the sub
    If txtFile.Text = "" Then Return

    ' create a stream reader object
    Dim sr As New IO.StreamReader(txtFile.Text)

    ' read the first line (file header) and do nothing to it to skip this line
    Dim line As String = sr.ReadLine()

    Dim data() As String            ' an array for holding data of a line
    Dim count As Integer = 0        ' a variable to count points of a route
    Dim lat() As Double             ' array for holding latitudes of a route
    Dim lon() As Double             ' array for holding longitudes of a route

    While Not sr.EndOfStream        ' use a while repetition block
        line = sr.ReadLine()        ' reads a line from the file
        data = line.Split(",")c     ' parse data from the line
        If data(1) = cboRoute.Text Then     ' data(1) is a route number
            lstOutput.Items.Add(data(3) & ", " & data(4))
            ReDim Preserve lat(count)       ' expands the lat array
            ReDim Preserve lon(count)       ' expands the lon array
            lat(count) = CDbl(data(3))      ' stores the latitude
            lon(count) = CDbl(data(4))      ' stores the longitude
            count += 1                      ' increments the counter
        End If
    End While
    sr.Close()              ' closes the stream reader

    ' display point count of the selected bus route
    txtInfo.Text = "Point count: " & CStr(count)
End Sub
```

The second line inside the procedure creates a stream reader object that opens the data file, and the third line reads a line of data from the data file. Nothing is done to this data line because it is a file header that contains no data. Then the program declares a few variables including two array variables, *lat* and *lon*, to hold coordinates of a selected route. Since we do not know the number of data points of a selected bus route beforehand, no upper bounds are specified to these arrays when they are declared. Inside the repletion block, once a data point is selected and its coordinates need to be preserved, the *lat* and *lon* arrays are expanded using keywords "ReDim Preserve" with a new size *count*. After storing the coordinates, variable *count* is incremented for use in next iteration.

Run the program and test the buttons. You will find that the process button does the exact same job as the "RouteCoordinates" project.

4) To compute bus route length, you will create a function (name it "RouteLength()") that takes an array of latitudes and an array of longitudes as input (parameters), performs length calculation, and returns a result. This function will need to call the EarthDistance() function you used in Example 2 of Chapter 4 to calculate the distance between each adjacent pair of geographic coordinates in the input arrays. So you will need to copy the code of function EarthDistance() (in file earthdist.vb in folder Source_Code on the CD-ROM) into this project. The RouteLength() function can be implemented as the following.

```
Private Function RouteLength(x() As Double, y() As Double) As Double
    Dim length As Double = 0
    For i As Integer = 0 To x.Length - 2
        length += EarthDistance(x(i), y(i), x(i + 1), y(i + 1))
    Next
    Return length
End Function

Private Function EarthDistance(ByVal lon1 As Double, ByVal Lat1 As Double,
                               ByVal lon2 As Double, ByVal Lat2 As Double) _
                           As Double
    ' ...
End Function
```

5) Use the following code to call the RouteLength() function after the repetition block (better after the stream reader is closed) in the process button's Click event procedure and report route length.

```
    ' call the RouteLength() function to compute route length
    Dim length As Double = RouteLength(lon, lat)

    ' display point count of the selected bus route
    txtInfo.Text = "Point count: " & CStr(count) &
                   ", Route length: " & Math.Round(length, 2) & " km."
```

The first parameter of the RouteLength() function is array *x*() and the second parameter is array *y*(). When the function is called, argument array *lon* is passed to *x* and array *lat* is passed to *y*. Please do not misplace the order of these arguments. Similarly, to call the EarthDistance() function in the RouteLength() function, please pay attention to the order of arguments. Arguments *x*(*i*), *y*(*i*) and *x*(*i*+1), *y*(*i*+1) make a pair of coordinates of two adjacent points along a bus route. These coordinate values are sequentially passed to parameters *lon1*, *lat1* and *lon2*, *lat2*. If no error is made, the project will produce a result similar to the screenshot of Figure 10.

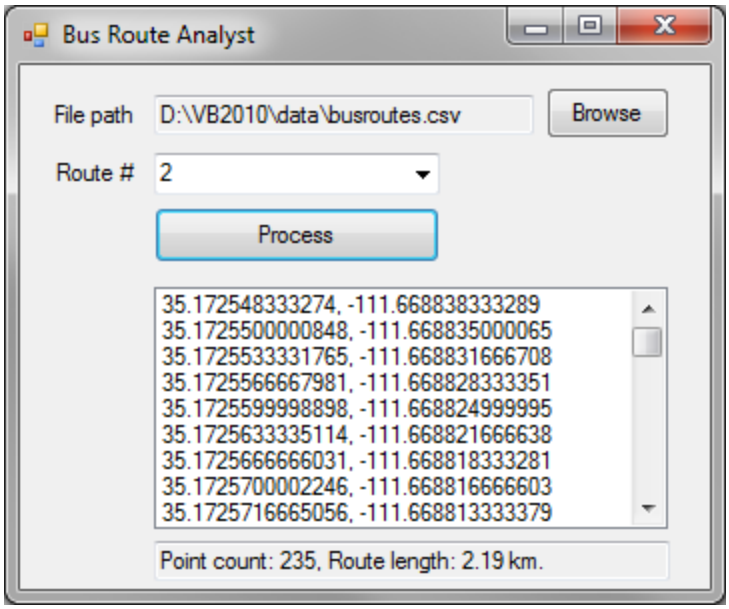


Figure 10 Execution result of project BusRouteAnalyst

7.2 StreamWriter

Like StreamReader, StreamWriter is also a class and it also exists in the IO namespace. StreamWriter works very similar to StreamReader. To use it, you must create an object from it. A stream writer object has a WriteLine() method that lets you write a line of text into a file. It also has a Close() method and a Dispose() method to perform clean up tasks.

Example 5: To demonstrate the usage of the StreamWriter class, please create a new VB project named "CopyFile" to copy the content of one text file to a new file. On the main form, please create two text boxes (txtFileIn, txtFileOut), a button that will be used to browse a text file into the input file box, and a button that will be used to copy the contents of the input file to a new file with name specified in the output file box.

Use the code below to implement this procedure. After creating the CopyFile() sub procedure, please call it in the Click event procedure of the "Copy File" button. When you test the program, make sure the output folder (e.g. "D:\Temp" or whatever you specify in text box txtFileOut) exists.

```
Private Sub CopyFile()
    If Not IO.File.Exists(txtFileIn.Text) Then Return
    If txtFileOut.Text = "" Then Return ' checks output file name

    ' create a stream reader object and open the input file
    Dim sr As New IO.StreamReader(txtFileIn.Text)
    ' create a stream writer object from the StreamWriter class
    Dim sw As New IO.StreamWriter(txtFileOut.Text)

    lstContent.Items.Clear()        ' clears the list box
    While Not sr.EndOfStream        ' loops until pointer moves to file end
```

```
            Dim str As String = sr.ReadLine ' reads a line from the input file
            sw.WriteLine(str)           ' writes the string into the output file
        End While
        sr.Close()      ' close the input file
        sw.Close()      ' close the output file
        sr.Dispose()    ' dispose the sr object
        sw.Dispose()    ' dispose the sw object
        MessageBox.Show("File successfully copied to " & txtFileOut.Text)
    End Sub
```

The first line in the procedure uses built-in function IO.File.Exists() to check if the input file exists. The function returns a Boolean value.

Summary and Highlights

An array is a variable that can hold a series of values of a type. Although an upper bound is optional at array variable declaration, an upper bound must be specified in order for the array to have a capacity. Because indexes of array elements start from 0, the length of an array, i.e. the number of values can be held by an array, is the upper bound + 1.

An array can be resized by keyword ReDim. But when ReDim is used alone, existing data in the array will be lost. To preserve existing data in an array while resizing it, keyword Preserve can be used together with ReDim.

As an object, an array has properties and methods. A very commonly used property is the “Length” property which tells the size of the array. Besides Length, the size of an array can also be obtained from the “Count()” method. Depending on data type, arrays can have various properties and methods. You can use the VB IntelliSense to find all of them. If an array is a numeric type, several statistical methods are available to perform basic statistics on values in the array.

VB provides a number of techniques for accessing text files. Function ReadAllLines() reads all lines of a text file and returns a String array object in which each element holds a line retrieved from the text file. Another technique to read text files is to use the StreamReader class to create a stream reader object. A stream reader object can read a text file line by line as needed.

If a text string contains multiple values that are separated by certain characters, these values can be parsed by calling the Split() method of the string object. By specifying a separator character or characters, the Split() methods returns a string array in which each element is a sub string value parsed from the host string.

In VB programming, you can use the OpenFileDialog class to create open file dialog objects. Then you can set file filter criteria to the Filter property of the open file dialog object to browse file of specified types. You can retrieve the full path of a user selected file from the FileName property of the object.

Exercises

1. Answer the following questions.
   1) What is an array?
   2) How do you declare an array?
   3) If an array's upper bound is n, how many elements can the array hold?
   4) If you want to create an array to hold 100 values, what upper bound should be specify?
   5) How do you declare an array and initialize its contents in one line?
   6) How do you get the number of elements in an array?
   7) How do you resize an array?
   8) What does keyword ReDim do?
   9) What does keyword Preserve do?
   10) What does built-in function ReadAllLines do?
   11) What is the namespace of built-in function ReadAllLines?
   12) What is the namespace of built-in function Round?
   13) What does the Split method of a string object do?
   14) How do you extract data from comma delimited text files?
   15) What does the Try block do?
   16) In a Try block, where do you put the code you want to try?
   17) What happens if a run-time error occurs in a Try block?
   18) How do you create an open file dialog object?
   19) How do you specify a file filter for an open file dialog object?
   20) How do you get the full path of a file selected by the user in an open file dialog box?
   21) How do destroy an open file dialog object?
   22) If an array object is passed to a parameter which is declared with keyword ByVal, can the array bring updated element values back to the client procedure?
   23) What does class StreamReader do?
   24) How do you create a stream reader object from class StreamReader class?
   25) How do you read a line by a stream reader object?
   26) What does the ReadLine method of the StreamReader class do?
   27) When you use a loop to read through a text file, how do you terminate the loop?
   28) What does the Close() method of StreamReader class do?
   29) What does class StreamWriter do?
   30) What method of the StreamWriter do you call to write a line into a text file?

2. Analyze the following programs and determine execution results

   1)

```
Private Sub btnStart_Click(...) Handles btnStart.Click
    Dim nums(3) As Integer
    nums(0) = 5
    nums(1) = 10
    nums(2) = 25
    nums(3) = 20
    lstOutput.Items.Add("Number of elements: " & nums.Length)
    lstOutput.Items.Add("Sum: " & nums.Sum())
End Sub
```

   2)

```
Private Sub btnStart_Click(...) Handles btnStart.Click
     Dim movies() As String = {"Gravity", "Spider-Man", "Star Wars"}
     lstOutput.Items.Add("Number of movies: " & movies.Count())
     lstOutput.Items.Add("My favorite movie is " & movies(1))
End Sub
```

3)

```
Private Sub btnStart_Click(...) Handles btnStart.Click
    Dim months() As String = {"January", "February", "March"}
    ReDim Preserve months(5)
    months(3) = "April"
    months(4) = "May"
    months(5) = "June"
    For i = 0 To months.Length - 1
        lstOutput.Items.Add(months(i))
    Next
End Sub
```

4)

```
Public Class Form1
    Private day() As String

    Private Sub btnStart_Click(...) Handles btnStart.Click
        GetDays()
        PrintDays()
    End Sub

    Private Sub PrintDays()
        For i As Integer = 0 To day.Length - 1
            If i Mod 2 = 1 Then
                lstOutput.Items.Add(day(i))
            End If
        Next
    End Sub

    Private Sub GetDays()
        ReDim day(6)
        day(0) = "Sunday"
        day(1) = "Monday"
        day(2) = "Tuesday"
        day(3) = "Wednesday"
        day(4) = "Thursday"
        day(5) = "Friday"
        day(6) = "Saturday"
    End Sub
End Class
```

5)

```
Private Sub btnStart_Click(...) Handles btnStart.Click
     Dim months() As String = {"January", "February", "March", "April",
                               "May", "June", "July", "August", "September",
                               "October", "November", "December"}

        For i = 0 To months.Length - 2 Step 2
            lstOutput.Items.Add(months(i) & " - " & months(i + 1))
        Next
    End Sub
```

6)

```
Private Sub btnStart_Click(...) Handles btnStart.Click
     Dim str As String = "Monday, Tuesday, Wednesday, Thursday, " &
                         "Friday, Satuday, Sunday"
    Dim days() As String = str.Split(","c)
    str = ""
    For i As Integer = 0 To days.Length - 1
        days(i) = days(i).Trim().Substring(0, 3).ToUpper()
        str &= days(i) & ", "
    Next
    str = str.Substring(0, str.Length - 2)
    lstOutput.Items.Add(str)
End Sub
```

7)

```
Private Sub btnStart_Click(...) Handles btnStart.Click
    Dim strNum As String = "1, 2, 3, 4, 5, 6, 7, 8, 9, 10"
    Dim vals() As Integer = GetValues(strNum)
    Dim sum As Integer = vals.Sum()
    lstOutput.Items.Add("Sum = " & sum)
End Sub

Private Function GetValues(strData As String) As Integer()
    Dim data() As String = strData.Split(","c)
    Dim num(data.Length - 1) As Integer
    For i As Integer = 0 To data.Length - 1
        num(i) = CInt(data(i).Trim())
    Next
    Return num
End Function
```

8)

```
Public Class Form1
    Private lines() As String = {"ID,X,Y", "1,10,12", "2,20,18"}

    Private Sub btnStart_Click(...) Handles btnStart.Click
        Dim x() As Double = Nothing
        Dim y() As Double = Nothing
        GetData(x, y)
        For i As Integer = 0 To x.Length - 1
            lstOutput.Items.Add("x = " & x(i) & ", y = " & y(i))
        Next
    End Sub

    Private Sub GetData(ByRef x() As Double, ByRef y() As Double)
        Dim data() As String
        Dim count As Integer = 0
        For i As Integer = 1 To lines.Length - 1
            data = lines(i).Split(","c)
            ReDim Preserve x(count), y(count)
            x(count) = CDbl(data(1))
            y(count) = CDbl(data(2))
            count += 1
        Next
    End Sub
End Class
```

3. Text file “grades.txt” in folder Data on the CD-ROM contains test grades of four subjects of a class.

```
Name, English, Math, Science, Social Studies
Adam, 75, 80, 90, 63
Betty, 90, 76, 78, 88
Chris, 95, 82, 83, 92
Duane, 80, 90, 94, 72
Elena, 98, 92, 88, 86
Fried, 80, 90, 95, 75
George, 63, 72, 85, 76
Hilary, 82, 86, 76, 88
Iris, 80, 85, 95, 80
Jeff, 86, 98, 94, 90
```

Create a new project named “Grades” to carry out the following tasks and display results in a list box.

1) Display the content when the user selects the file by clicking a browse button
2) Calculate the average grade of each student
3) Calculate the average of each subject of the class
4) Display “A” students in math (grade ≥ 90)
5) Display “B” students in math (80 ≤ grade < 90)